



Phage
SECURITY

UpShot

Security Review

Conducted by:

Pyro, Lead Security Researcher
YanecaB, Security Researcher
yovchev_yoan, Security Researcher

13.02.2026

Table of Contents

About Phage Security 3

Disclaimer 3

System overview 3

Executive summary 3

 Overview 3

 Timeline 4

 Scope 4

 Issues Found 4

 Findings Summary 4

Findings 5

 High Severity 5

 [H-01] Deposit Nonce Can Be Replayed 5

About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to [Pyro](#).

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

[UpShot](#) contracts serves as a USDC treasury that uses backend signed EIP712 authorizations to gate player deposits and supports admin controlled (optionally multisig approved) withdrawals, enabling the game to securely manage on chain funds tied to its off chain platform.

Executive summary

Overview

Project Name	UpShot
Repository	private
Commit hash	d76e8a41a293b00e1a5ae82ef8731766736d5fdf
Remediation	a52bf138a853b77e819d7c1f0090c5b2ec17ac48
Methods	Manual review



Timeline

Audit kick-off	12.02.2026
End of audit	12.02.2026
Remediations start	13.02.2026
Remediations end	13.02.2026

Scope

contracts/PlatformTreasury.sol

Issues Found

Severity	Count
High	1
Medium	0
Low	0

Findings Summary

ID	Title	Severity	Status
[H-01]	Deposit Nonce Can Be Replayed	High	Fixed

Findings

High Severity

[H-01] Deposit Nonce Can Be Replayed

The `deposit()` function does not verify nonce uniqueness or ownership. Any user can reuse any nonce, enabling two attack vectors:

1. **Front-Running:** Attacker monitors mempool, copies victim's nonce and front-runs their deposit transaction. Backend rejects the attacker's deposit (wrong depositor), but tokens remain locked in contract as orphaned deposits.
2. **Griefing:** Attacker repeatedly front-runs users with minimum deposits (1 USDC) using their nonces, disrupting legitimate deposits while accumulating orphaned funds in the contract.

```
function deposit(uint256 amount, bytes32 nonce) external {
    if (nonce == bytes32(0)) revert InvalidNonce();
    // No uniqueness check, no ownership verification
    IERC20(usdc).safeTransferFrom(msg.sender, address(this), amount);
}
```

Impact: Cheap griefing attacks, orphaned deposits, accounting mismatches between contract and backend.

Recommendation

Implement signature-based authorization to bind nonces to specific users and amounts.